

07 · LLM Reasoning Agent

2026-07-31

Contents

07 · LLM Reasoning Agent	1
1. 核心职责	1
1.1 设计原则	2
2. Prompt 策略	2
2.1 Few-shot 示例	2
2.2 负例约束	3
3. 推理范式	3
3.1 Chain-of-Thought (CoT)	3
3.2 Tree-of-Thoughts (ToT)	3
3.3 Self-Reflection	4
3.4 Self-Consistency Voting	4
4. 工具调用协议	4
4.1 工具调用闭环	5
4.2 工具结果格式化	5
5. 上下文管理	5
5.1 上下文预算	5
6. 错误恢复	5
7. 输出格式	6
8. 设计路线小结	6

07 · LLM Reasoning Agent

本文档阐述 LLM Reasoning Agent 的核心职责、Prompt 策略、推理范式(CoT/ToT/Reflection/Voting)、工具调用协议、上下文管理与错误恢复。Agent 在已验证的 Geometry Graph/DSL 之上进行证明规划，协调 Verifier 与 Solver 形成闭环。

1. 核心职责

LLM Reasoning Agent 是推理层中枢，职责：

- 理解题目**：将题目文字、DSL、目标统一编码进 Prompt。
- 定理检索 (RAG)**：从几何定理库召回相关定理。
- 证明规划**：将”求证/求解目标”分解为子目标序列。
- 假设生成**：提出候选中间结论，交 Verifier/Solver 验证。
- 反思修正**：验证失败时分析原因，调整规划。

6. **步骤生成**：将验证通过的推理链组织为可读证明。

1.1 设计原则

- **仅基于已验证事实推理**：不得使用未验证关系作为前提。
 - **规划与求解分离**：LLM 负责规划，Solver 负责精确计算。
 - **闭环验证**：每个中间结论必须调用 verify/solve。
 - **可追溯**：每步推理记录依据与工具调用结果。
-

2. Prompt 策略

采用**结构化多段 Prompt + 工具调用**组合：

[System]

你是一名几何证明专家。你只能基于已验证的几何事实推理。

可调用的工具：verify(relation), solve(equation), search(theorem), reflect(), graph_query(q)。

任何未经验证的断言不得进入结论。证明需逐步给出，每步附理由。

[Context]

Geometry DSL

{dsl}

已验证关系(列表)

{verified_relations}

定理库片段(RAG 检索 Top-K)

{retrieved_theorems}

题目

{problem_text}

[Task]

目标: {goal}

[Instructions]

1. 先列出已知与目标。
2. 检索相关定理，提出证明规划(子目标树)。
3. 对每个子目标调用 verify/solve 验证。
4. 验证失败则 reflect 并调整规划(最多 3 轮)。
5. 汇总为完整证明，每步注明依据(定理/已验证关系/计算)。
6. 输出最终答案与置信度。

2.1 Few-shot 示例

提供 2~3 个高质量证明范例(按题型: 三角形/圆/椭圆), 覆盖”相似 → 比例 → 结论”等典型模式。示例存放于 prompts/fewshot/, 按题型动态选取。

2.2 负例约束

明确约束：- “不得使用未验证关系” - “不得跳步” - “不得编造数值（必须调用 solve 计算）” - “若 verify 返回 false，必须 reflect 修正，不得强行使用”

3. 推理范式

范式	用途	实现	适用场景
Chain-of-Thought (CoT)	线性证明生成	默认主路径	简单题、单链证明
Tree-of-Thoughts (ToT)	多分支证明搜索	子目标分叉，每分支独立验证，剪枝	复杂题、多路径
Self-Reflection	失败回溯	Verifier 返回 false 时触发	任何验证失败
Self-Consistency Voting	多解集成	采样 N 条路径投票	高 stakes 题、答案分歧

3.1 Chain-of-Thought (CoT)

LLM 线性生成证明步骤，每步调用 verify 确认。适用于路径明确的题。

已知：A,B,C,E 共圆；D 为 BC 中点

目标： $AB \cdot AC = AD \cdot AE$

步骤1：提出证 $\triangle ABE \sim \triangle ACD \rightarrow \text{verify}(\angle ABE = \angle ACD) \checkmark$

步骤2： $\text{verify}(\angle BAE = \angle CAD) \checkmark$ （公共角）

步骤3：得相似 $\rightarrow \text{solve}(\text{比例}) \rightarrow AB/AC = AE/AD$

步骤4：变形 $\rightarrow AB \cdot AC = AD \cdot AE \checkmark$

3.2 Tree-of-Thoughts (ToT)

对目标生成多个证明分支，每分支独立验证，剪除失败分支，保留成功路径。适用于多种证明思路并存的题。

目标：Prove $AB \cdot AC = AD \cdot AE$

├─ 分支1：证 $\triangle ABE \sim \triangle ACD$

| └─ 子1.1： $\text{verify}(\angle ABE = \angle ACD) \rightarrow$ 同弧圆周角 $\checkmark$

| └─ 子1.2： $\text{verify}(\angle BAE = \angle CAD) \rightarrow$ 公共角 $\checkmark$

| └─ 相似成立 $\rightarrow$ 比例 $\rightarrow$ 结论 $\checkmark$ [采纳]

├─ 分支2：用圆幂定理

| └─ $\text{verify}(D \text{ 圆幂}) \rightarrow D \text{ 不在圆上 } \times$ [剪枝]

└─ 分支3：坐标法

└─ $\text{solve}(\text{解析几何}) \rightarrow$ 可行但繁琐，置信低 [备选]

ToT 算法：

```
def tot_search(goal, graph, depth=0, max_depth=8):
    if goal_achieved(goal, graph):
        return extract_proof(graph)
    if depth > max_depth:
        return None
    for sub_goal in propose_subgoals(goal, graph):
```

```

result = verify_or_solve(sub_goal, graph)
if result.verified == "true":
    graph.add_derived(sub_goal, result)
    proof = tot_search(goal, graph, depth+1)
    if proof: return proof
    graph.retract(sub_goal)
# false/uncertain: 剪枝或降级
return None

```

3.3 Self-Reflection

当 Verifier 返回 false, LLM 触发 reflect: - 分析失败原因 (关系不存在 / 定理用错 / 规划偏差)。- 生成修正后的新规划。
- 限制最大反思轮次 (3 轮), 避免死循环。

```

def reflect(failure, plan, history):
    prompt = f"""
    上一步失败: {failure}
    当前规划: {plan}
    历史: {history}
    分析失败原因并给出修正规划。
    """
    return llm(prompt)

```

3.4 Self-Consistency Voting

对同一题, 以不同温度 (如 0.3/0.7/0.9) 采样 N (如 5) 条独立证明路径, 对最终答案投票: - 多数一致 → 高置信。- 分歧 → 触发更深搜索或标记” 需人工复核”。

```

def self_consistency(problem, n=5):
    paths = [run_reasoning(problem, temp=t) for t in sample_temps(n)]
    answers = [p.answer for p in paths]
    answer, count = most_common(answers)
    confidence = count / n
    if confidence < 0.6:
        return escalate_to_human(paths)
    return merge_proofs(paths, answer, confidence)

```

4. 工具调用协议

Agent 通过函数调用 (tool calling) 与外部模块交互:

```

tools = {
    "verify": verify_relation, # (relation, src, dst, attrs) -> VerifyResult
    "solve": symbolic_solve, # (equations) -> Solution
    "search": theorem_search, # (query) -> list[Theorem]
    "reflect": self_reflect, # (failure) -> revised_plan
    "graph_query": graph_query, # (query) -> subgraph/facts
    "construct": construct_aux, # (object_desc) -> new node (辅助构造)
}

```

4.1 工具调用闭环

LLM 每提出一个中间结论，必须调用 verify 或 solve，结果回填上下文。这把 LLM 的”自由发挥”约束在”可计算验证”的轨道内。

```
LLM: "我认为  $\triangle ABE \sim \triangle ACD$ "
→ call verify("Similar", "Triangle_ABE", "Triangle_ACD")
← {"verified": "true", "evidence": " $\angle ABE = \angle ACD$ ,  $\angle BAE = \angle CAD$ "}
LLM: "由相似得  $AB/AC = AE/AD$ "
→ call solve(["AB/AC = AE/AD", "goal: AB*AC = AD*AE"])
← {"solution": "AB*AC = AD*AE 成立"}
```

4.2 工具结果格式化

工具结果以结构化 JSON 返回，LLM 易解析。失败结果含 reason 字段，供 reflection 使用。

5. 上下文管理

几何题证明可能较长，需管理上下文：

策略	用途
DSL 紧凑模式	省略坐标，仅保留拓扑，减少 token
历史压缩	已验证子目标摘要化，不再展开细节
分段记忆	按子目标分段，每段独立
工具结果缓存	重复查询复用结果

5.1 上下文预算

- System + Tools: ~1k token
- DSL + 已验证关系: 2~4k token
- 定理 RAG: 1~2k token
- 推理历史: 动态，超 8k 触发压缩
- 总预算: 16~32k token (依模型)

6. 错误恢复

错误	恢复策略
verify 返回 false	触发 reflect，调整规划
solve 无解	检查方程构建是否错误，reflect
工具调用格式错	重试 + 格式提示
上下文超限	压缩历史
反思超 3 轮	返回”无法证明” + 已知部分结论
多路径分歧	触发人工复核

绝不编造原则：任何无法验证的结论不得进入最终证明。宁可返回“无法证明”也不编造。

7. 输出格式

```
{
  "answer": "AB · AC = AD · AE 成立",
  "confidence": 0.93,
  "proof": [
    {"step":1,"statement":"A,B,C,E 共圆 (外接圆)","reason":"已知","verified":true},
    {"step":2,"statement":"∠ABE = ∠ACE","reason":"同弧圆周角定理","verified":true,
      "tool_call":{"name":"verify","args":["Equal","∠ABE","∠ACE"]}},
    {"step":3,"statement":"△ABE ∽ △ACD","reason":"两角对应相等","verified":true},
    {"step":4,"statement":"AB/AC = AE/AD","reason":"相似对应边成比例","verified":true,
      "tool_call":{"name":"solve","args":["AB/AC=AE/AD"]}},
    {"step":5,"statement":"AB · AC = AD · AE","reason":"比例变形","verified":true}
  ],
  "reasoning_path": "CoT(分支 1)",
  "reflection_count": 0,
  "voting": {"n":5,"agree":4,"confidence":0.8}
}
```

8. 设计路线小结

LLM Reasoning Agent 的设计路线为：“结构化多段 Prompt(系统/上下文/任务/指令) → Few-shot + 负例约束 → 多范式推理 (CoT/ToT/Reflection/Voting) → 工具调用闭环 (verify/solve/search) → 上下文管理 (压缩/预算) → 错误恢复 (绝不编造)”。它作为推理中枢，将 LLM 的规划能力与 Verifier/Solver 的精确性结合，形成“假设—验证—求解”的可靠闭环。